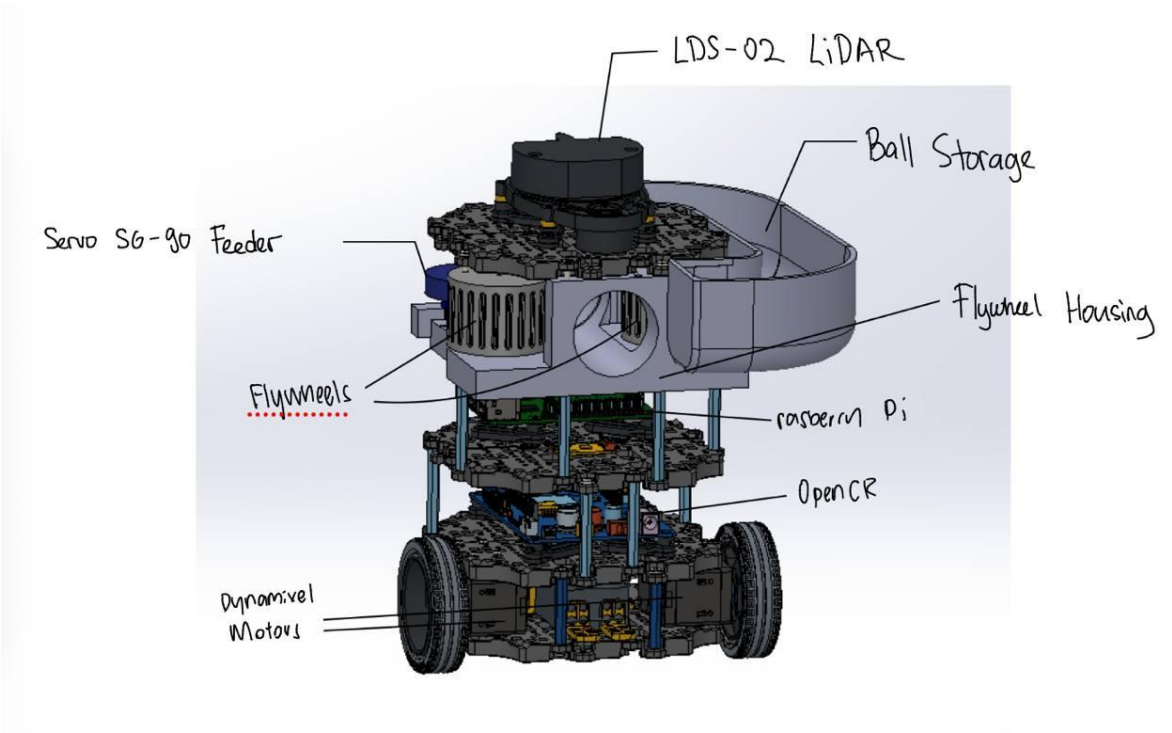
END-USER DOCUMENTATION

Autonomous Mobile Robot for Smart Warehouse Intralogisticst

Section 1 — General System Description & Critical Data

A TurtleBot3 Burger-based AMR with a custom launcher payload. Mission: autonomously navigate an unknown warehouse, detect ArUco-marked stations, and deliver ping-pong balls into static (Station A) and oscillating (Station B) receptacles within 25 minutes. Bonus capability: elevator traversal via REST API to extend exploration to Level 2.



Turtlebot with Custom Payload CAD Model

Parameter	Value
Model / Platform	TurtleBot3 Burger + custom payload (ramp, launcher, camera)
Robot Identifier	Group 1 – Russell, Shandrico, Alex, Moksh
Software Version	v1.0.0
Total Mass	1402.24 g (1.40224 kg)
Overall Dimensions (L × W × H)	138 mm × 178 mm × 192 mm
Centre of Gravity — X (back +)	−56.7 mm from ball-caster origin
Centre of Gravity — Y (up)	95.3 mm from ground
Centre of Gravity — Z (left +)	2.8 mm from ball-caster origin
Battery	Li-Po 11.1 V, 1800 mAh (84.03 g)
Compute	Raspberry Pi 4 + OpenCR 1.0
Sensors	LDS-02 LiDAR, RPi Camera Module V2 (8 MP, IMX219)
Drive	2 × Dynamixel XL430-W250
Launcher	Dual counter-rotating flywheels (2 × RC-300 / PEL00882 DC motor) + SG90 servo gate
Ball Storage Capacity	9 ping-pong balls (curved gravity-feed tube)
Mission Window	25 minutes
Purpose	Autonomous warehouse intralogistics: explore unknown space, detect ArUco beacons, deliver ping-pong balls to static and dynamic receptacles, optional elevator traversal to Level 2

Section 2 — Technical Guide

Subsystems: Navigation (Dynamixel XL430 drive motors + SLAM), Sensors (LDS-02 LiDAR + RPi Camera V2 for ArUco detection), Computation (RPi 4 for ROS 2 / vision + OpenCR 1.0 for motor control / IMU), Launcher (gravity-feed tube, dual counter-rotating flywheels, SG90 servo gate).

Mission Flow (6 phases):

- **Phase 1: System Initialization:** Upon power-on, the robot automatically boots up its sensors and begins generating a live 2D map of the warehouse using its laser scanner (LiDAR).
- **Phase 2: Initial Systematic Sweep:** Before moving deep into the facility, the robot performs a structured, outward-expanding search pattern to map out as much of its immediate surroundings as possible.
- **Phase 3: Autonomous Target Search:** The robot safely navigates through the newly discovered areas of the warehouse, automatically detecting and avoiding obstacles in real-time to locate the specific delivery stations.
- **Phase 4: Comprehensive Map Verification:** To guarantee no areas were missed during the initial search, the robot conducts a final, exhaustive sweep to ensure 100% of the accessible warehouse floor has been securely mapped.
- **Phase 5: Static Delivery (Station A):** Using its onboard camera, the robot visually identifies the docking marker at Station A. It aligns itself perfectly and dispenses 3 ping-pong balls sequentially, pausing briefly between shots to ensure accuracy.
- **Phase 6: Dynamic Delivery (Station B):** The robot tracks the moving target at Station B. It calculates the speed and timing required to shoot the balls accurately into the moving receptacle.
- **Phase 7: Level 2 Access (Elevator):** The robot identifies the elevator lobby, digitally calls the lift via a wireless network request, and safely relocates itself on the second floor to continue its mission.
- **Phase 8: Mission Logging:** Throughout the operation, the robot securely records all system data to allow for post-mission review and performance analysis.

Key Specification:

High-Precision Mapping: The robot maps the warehouse with an accuracy of within 5 cm, ensuring safe and exact navigation even in tight spaces.

Long-Range Station Detection: The onboard vision system can detect delivery stations from up to 2.2 meters away.

Fast Reflexes: The robot reacts to unexpected obstacles (like dropped boxes or walking personnel) in under a tenth of a second (< 100 ms).

Advanced Target Tracking: The system uses predictive tracking to ensure it doesn't lose sight of moving delivery targets

Software Setup:

This setup process assumes ros2 Humble environment is already installed (follow the setup process at https://emanual.robotis.com/docs/en/platform/turtlebot3/quick-start/#pc-setup)

1. **Backup and delete your ros src folder:**
   cp ~/[YOUR_ROS2_WORKSPACE]/src ~/[YOUR_DESIRED_PATH]
   rm- rf ~/[YOUR_ROS2_WORKSPACE]/src

2. **Clone our GIthub repository into your ros2 workspace:**
   git clone https://github.com/Russell501/CDE2310_Group1.git

3. **Install ros2 dependencies and rebuild workspace:**
   rosdep install –from-paths src –ignore-src -r -y
   colcon build –symlink-install
   source install/setup.bash

Deployment:

1. **Turn on Turtlebot3**

2. **SSH into Turtlebot3**

3. **Run launch file in SSH Terminal:**
   python3 main_launch.py

4. **Run the GUI in another terminal (your own terminal):**
   ros2 run auto_nav mission_gui

5. **Run the launch file in another terminal (your own terminal):**
   ros2 launch auto_nav bringup_all.launch.py

6. **View the movement of the robot via the Rviz2 window, and the mission progress via the GUI window**

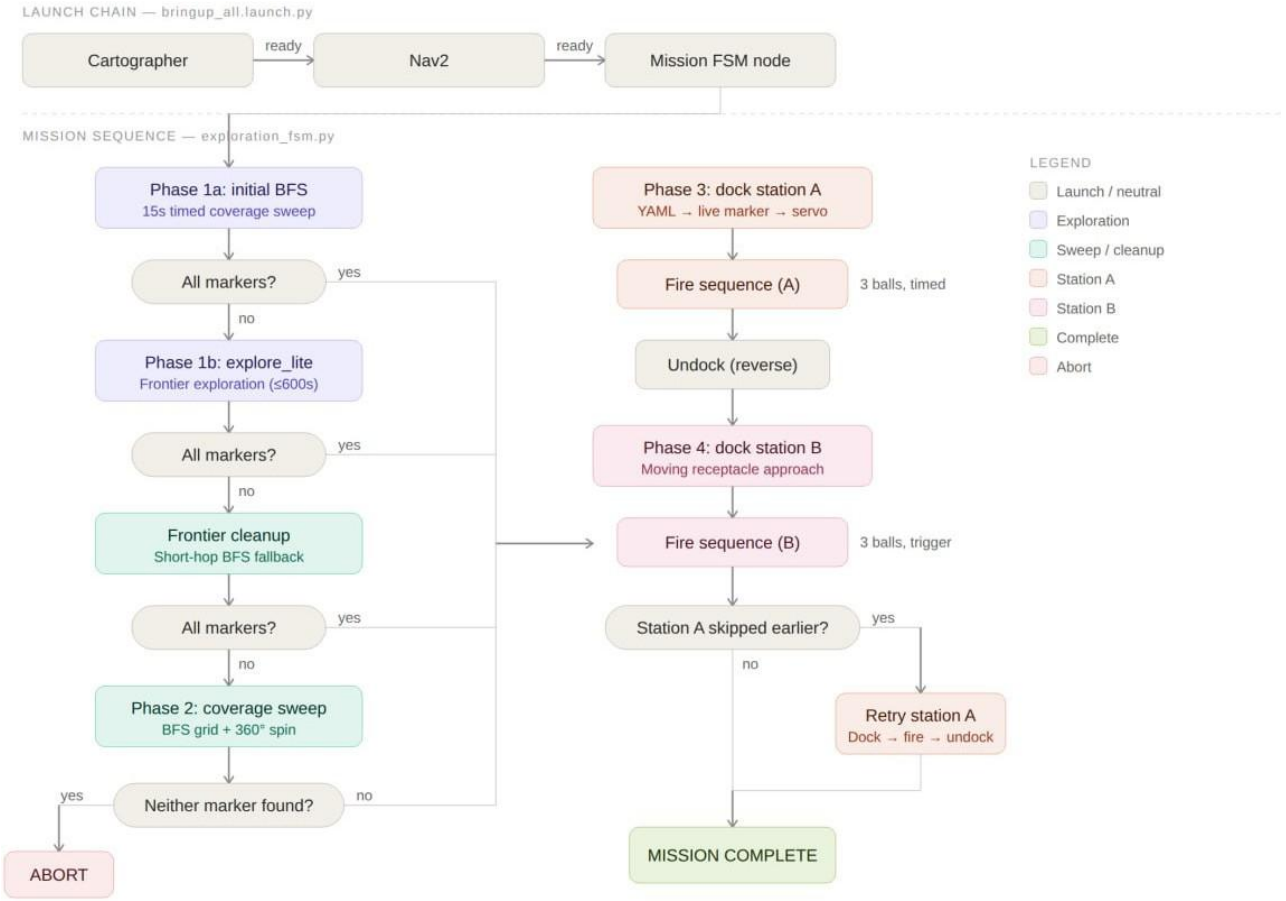
Troubleshooting steps:

1. **Missing base_link or odom topics from Rviz2 map:**
   Wait at least 30s for the topics to be published. If still unavailable, then terminate the launch files and run them again.

2. **Servo not moving to fire the balls:**
   First check the logs in the mission GUI for ball launch messages, and that they were successfully received in the SSH terminal. If all these are present, then check that all 3 wires from the servo are connected.

3. **Flywheel(s) not spinning when launching:**
   Like the previous issue, first check for logs in the mission GUI and SSH terminal. Then check if the flywheel is rubbing the top waffle plate; if so, gently press it back down. Lastly, check all connections from the motors to the motor driver, and from the motor driver to the raspberry pi.

Section 3 — System Architecture



Section 4 — Acceptable Defect Log

Items below are known imperfections that have been judged acceptable for mission operation. They are documented here for the operator's awareness. (No defects have been recorded by the team in the source materials provided. This log is to be populated during build and integration in Weeks 8–9.)

ID	Description	Mission Impact	Operator Note
1	L298N missing one standard motor driver screw	None	Ensure wires are secured and not loose

Section 5 — Factory Acceptance Test (FAT)

Pre-mission checklist of factory checks. All items must be ticked off and signed before the robot is presented for the final mission run.

#	Check Item	Pass / Fail	Date / Initials
1	Battery 11.1V / 1800 MAh charged & load verified		
2	All frame layers & payload secured		
3	LiDAR 360 degree FOV unobstructed		
4	Both Dynamixel motors respond to test commands		
5	Flywheel motors spin freely, SG90 gate cycles without jam		
6	Storage tube loaded with 9 balls; gravity feed confirmed		
7	Camera: 640*480@30 fps; ArUco detection active		
8	SLAM: Cartographer launches & maps on power-on		
9	Explore_lite: frontiers identified & goals issued		
10	Nav2 DWB: obstacle response < 100ms		
11	Docking: ±2cm lateral, 0.2m distance, ±3 degree angular		
12	Station A: 3 ball sequence bench tested		
13	Station B: Latency compensation rig tested		
14	Lift API: HTTP POST/GET round trip confirmed		
15	ROSBAG recording starts on launch		
16	RPi Voltage >= 4.75V under worst case load		
17	Motor & Signal wires routed separately; Caps fitted		
18	RPi and L298N not in contact		
19	E-Stop functional		
20	Document Pack Ready		

Section 6 — Maintenance & Part Replacement Log

Running record of maintenance operations and replacements of critical components on this robot. (No maintenance events have been recorded in the source materials provided. This log is to be populated as the robot is built, tested and serviced.)

Date	Component	Action / Reason	By
19/03/2026	RPI Camera	RPI Camera was not working, even after testing with different RPI and flexible flat cable (FFC)	Russell Ng
07/04/2026	11.1V Li-ion Battery	Battery cable broken	Alex